

交叉残差图神经网络用于生物分子图分类： 残差路由的实证研究

付小琴*

摘要

研究背景。虽然跳跃连接和层次化池化已被广泛用于保存图神经网络中的多深度信息，但现有工作均未直接分离出跨分支隐藏状态交换或跨块图摘要重新注入是否优于普通同路径残差复用。这正是本文所填补的研究缺口。

研究方法。本文提出交叉残差图神经网络（CR-GNN）作为一个受控比较框架，涵盖五种信息流拓扑——普通堆叠、节点级残差复用、节点级交叉交换、图级残差传播和图级交叉摘要交换——在共享主干网络和分层五折验证下进行评估。通过门控机制、残差路由和参数扫描等针对性消融实验，结合跨隐藏层的状态相似性和梯度范数深度扩展分析，分离出所观察到的性能模式背后的机制。

研究结果。在 CR-GNN 家族中，图级残差传播最频繁地取得最高峰值准确率，而节点级交叉残差交换在大型稀疏蛋白质图与注意力算子的组合上形成统治。交叉残差架构展现出最强的跨数据集排名一致性，且与稀疏路由结合时构成最频繁的最优配置。深度扩展分析进一步揭示了一个反直觉的模式：深层表示连续性最弱的架构却取得了最多的优胜次数，挑战了更强的层级稳定性必然预测更好性能的假设。

研究结论。本研究发现，更强的层级表示连续性并不必然预测更好的图分类效果——这一反直觉的发现我们称之为 CKA-性能悖论。交叉残差架构通过跨并行分支交换信息，成为跨不同数据集最一致可靠的性能表现者，其与稀疏路由的组合在大多数评估目标上构成最优配置。最后，双分支节点级交换在大型稀疏蛋白质图上尤为有效，为这一设计指明了一个合适的应用场景。

*柳州铁道职业技术学院铁路机车与车辆学院，广西柳州 545000

目录

1	引言	3
1.1	主要贡献	4
2	材料与方法	5
2.1	问题定义与符号	5
2.2	共享消息传递与图级学习	5
2.3	CR-GNN 架构家族	6
2.3.1	单分支和节点级残差变体	6
2.3.2	图级残差传播	8
2.4	残差路由机制	9
2.5	数据集	9
2.6	比较方法与主干算子	10
2.7	训练配置与实现细节	11
2.8	实验设计	11
2.9	评估指标与统计报告	12
3	结果	12
3.1	整体基准性能	12
3.2	各种复用拓扑分别在何时发挥作用?	13
3.3	机制分析: 表示连续性与性能之间的张力	13
3.4	量化性能权衡	16
3.5	探索性敏感性分析	16
4	讨论	17
4.1	为什么表示连续性最弱的架构获胜最频繁?	17
4.2	何时应选择交叉残差而非普通残差?	17
4.3	路由策略在架构选择之外是否重要?	18
4.4	局限性	18
4.5	未来方向	18
5	结论	19
6	致谢	20

1 引言

图神经网络 (GNN) 现已成为从分子图和蛋白质图中学习的标准方法, 因为它们能够将局部邻域结构与高阶关系上下文联合编码 (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017; Wu et al., 2021; Zhou et al., 2020)。在生物分子图分类中, 这种组合之所以重要, 是因为预测证据可能同时依赖于局部生化基序和更广泛的图级组织。因此, PROTEINS、DD、ENZYMES、MUTAG、AIDS 和 Mutagenicity 等基准测试套件仍作为方法导向图分类研究的受控测试平台发挥作用, 而非下游生物学发现的直接代理。

主要的方法论困难在于: 在这些设定下, 更深层的消息传递并非自动有益。重复传播会通过过度平滑和相关的深度效应 (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020b) 抑制具备信息量的中间状态, 而过度挤压效应 (Alon and Yahav, 2021; Topping et al., 2021) 则导致来自远距离节点的信息被压缩为无信息量的瓶颈。残差连接、归一化和正则化缓解了部分退化 (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019; Chen et al., 2020a), 但大多数残差设计仍沿同一分支复用信息。因此, 它们改善了优化过程, 却未直接检验替代的信息流拓扑是否能保存更有用的历史信号。

相关图学习工作已探索过跳跃连接、稠密聚合、APPNP 式传播、Jumping Knowledge 读出和层次化池化 (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019; Ying et al., 2018)。这些方法保存了来自多个深度或跨越池化层次的信息, 但它们并未直接分离出跨分支交换隐藏状态或跨块边界重新注入池化图摘要是否优于普通同路径残差复用。此前尚无研究在受控协议下直接比较这些复用拓扑——这正是本文所做的工作。

本文研究交叉残差图神经网络 (CR-GNN), 这是一个受控模型家族, 在共享的图分类主干下对比了普通堆叠、节点级残差复用、节点级交叉残差交换、图级残差传播和图级交叉摘要交换。研究围绕三个具体研究问题展开:

1. **交叉残差交互何时有帮助?** 在何种数据集特性和任务条件下, 跨并行分支交换隐藏状态能够比普通同路径复用更好地改善图分类?
2. **普通残差复用何时依然保持更强的默认选择?** 在哪些生物分子基准测试中, 更简单的单分支残差设计持续优于交叉残差替代方案? 这些数据集的结构特性如何解释这一模式?
3. **残差路由设计如何改变答案?** 超越“是否存在额外复用路径”的二元问题, 不同的过滤策略——可学习稠密路由、Top-k 选择和稀疏抑制——如何影响残差与交叉残差架构的相对性能?

围绕这三个问题组织研究对本文的方法论范围至关重要。目标不是宣称某一条额外通路总能产生更好的 GNN, 而是揭示残差路由何时以及如何发挥作用——即在主干网

络、数据划分和训练协议受控的条件下，哪种复用拓扑能够可靠地保存具有判别力的历史结构信号。

本文作为一项生物分子图分类方法学研究展开。主要证据来自以 PROTEINS、DD 和 ENZYMES 为核心的蛋白质导向基准测试，辅以补充的分子鲁棒性数据集。该设计使论文贴近具有生物学意义的图设定，同时保持可复现和受控的基准范围。

1.1 主要贡献

本文的主要贡献如下：

- **揭示设计权衡的受控比较框架。**本文并非提出单一新架构并宣称其普遍更优，而是在相同主干网络、数据划分和训练协议下比较了五种信息流设计——普通堆叠、节点级残差复用、节点级交叉残差交换、图级残差传播和图级交叉摘要交换。在 CR-GNN 家族 (GCNConv) 的 24 个数据集-算子组合中，该框架揭示了没有任何单一拓扑在所有维度上占优：GraphResGNN 在 50% 的组合中获胜，但在深度表示连续性上最弱；在 CR-GNN 家族内，NodeCrossGNN 赢得 25% 的组合，并在 DD 上跨四分之三的算子取得统治地位；GraphCrossGNN 是五种拓扑中排名最一致的设计。该受控设置使得我们可以识别在何种条件下选择哪种复用策略，而非仅问哪一个在平均意义上胜出。
- **系统性能权衡的实证证据。**在六个生物分子数据集和四种算子的分层五折评估下，GraphResGNN 在最多组合中取得最高峰值准确率，但其优势并非普遍存在：它在 MUTAG 上排名第五。NodeCrossGNN 是第二强的架构，尤其在大型稀疏蛋白质图 (DD，四种算子中的三种下获胜) 和基于注意力的配置 (GATConv) 上表现出色。GraphCrossGNN 虽从未排名第一，却是跨所有数据集最一致的性能表现者 (排名标准差 0.75)，始终位于前一半。深度扩展分析 ($L = 1-8$) 进一步揭示峰值准确率与表示连续性并不一致：NodeResGNN 在深层实现了最高的 CKA ($L = 8$ 时为 0.968)，但跨数据集表现出最大的性能波动。
- **深度扩展的机制分析。**通过从 $L = 1$ 到 $L = 8$ 层的逐层 CKA、余弦相似性和梯度范数追踪，本研究提供了性能模式背后的机制级证据。NodeResGNN 跨深度保持最强的表示连续性，但这并未转化为更优的峰值准确率。GraphResGNN 尽管深层 CKA 最低，却实现了最频繁的优胜——这表明当配合图级上下文重新注入时，层间的激进表示变化可以是一种生产性而非破坏性的设计特征。GraphCrossGNN 在性能排名和表示连续性方面均占据一致的中等位置，使其成为数据集特性不确定时最安全的选择。
- **具有实践指导意义的路由级机制分析。**超越架构标签，本研究考察了残差信息在复用前如何被过滤——在相同的五折协议下比较了可学习稠密路由、Top-k 通道选择和稀疏抑制。稀疏路由在四个代表性路由目标中的三个上最优，其中两个涉

及交叉残差架构——表明一种协同效应。路由策略被证明是目标依赖的。这为实践者提供了可操作的指导：架构和路由的选择应与数据集特性匹配，对于大型稀疏图和基于注意力的算子推荐交叉残差加稀疏路由，对于全局上下文占主导的数据集推荐图级残差加可学习路由。

2 材料与方法

本节定义共同的图分类设定、CR-GNN 架构家族、基准测试集以及共享的实验协议。指导原则是隔离信息复用的位置，同时保持其余流程尽可能一致。该设计选择对本文的方法论论证至关重要：如果主干算子、池化和评估协议保持固定，那么性能差异更可能归因于复用拓扑本身，而非无关的实现变化。

2.1 问题定义与符号

令 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 表示无向或有向图，其中 $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ 是包含 n 个节点的节点集， $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ 是边集。每个节点 $v_i \in \mathcal{V}$ 关联一个特征向量 $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$ ，整张图的节点特征收集于 $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ 中。图结构以邻接矩阵 $\mathbf{A}$ 表示。

给定训练数据集 $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$ ，目标是学习一个映射函数 $f_\theta : \mathcal{G} \rightarrow \hat{y}$ ，用于预测未见图的标签。本研究在同一图分类流程下比较五种架构：

- **PlainGNN**：无显式状态复用的顺序消息传递
- **NodeResGNN**：单分支节点级残差复用
- **NodeCrossGNN**：双分支节点级交叉交换
- **GraphResGNN**：跨顺序块的图级残差传播
- **GraphCrossGNN**：跨配对块的图级交叉摘要交换

该比较旨在隔离信息复用的位置：在单分支内部、跨并行分支之间，或通过跨块传递的池化图摘要。这隔离出架构家族背后一个具体的研究问题。同分支残差复用主要检验重新注入历史状态是否能稳定更深层的传播，而交叉残差设计则检验跨并行流交换信息是否能保存普通堆叠下可能丢失的互补结构。

2.2 共享消息传递与图级学习

令 Φ 表示一个消息传递算子。对于选定的主干网络，节点表示通过 L 层更新如下：

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_\Phi \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right). \quad (1)$$

在 L 层之后，读出函数 R 将节点级表示聚合为图嵌入：

$$\mathbf{h}_G = R\left(\left\{\mathbf{h}_i^{(L)} : i = 1, \dots, n\right\}\right). \quad (2)$$

分类器产生：

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}}\mathbf{h}_G + \mathbf{b}_{\text{clf}}), \quad (3)$$

模型通过最小化交叉熵损失进行端到端训练：

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}). \quad (4)$$

2.3 CR-GNN 架构家族

CR-GNN 被定义为一个紧凑的图分类架构家族。所有变体共享相同的输入投影、堆叠消息传递层、图级池化和线性分类器；它们仅在中间节点状态和图级摘要如何在深度和分支之间复用的方式上有所不同。该设计特意避免引入单独的编码器、任务特定特征工程或架构特定的读出头部，因为此类添加将使判断任何观察到的增益来自残差拓扑还是来自额外模型容量变得更加困难。

令 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 为一个图，其节点特征矩阵为 $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ ，邻接结构为 $\mathbf{A}$ 。模型可被实例化为四种标准消息传递算子之一：

$$\Phi \in \{\text{GCNConv}, \text{GATConv}, \text{SAGEConv}, \text{GINConv}\},$$

并在一个模型实例内统一使用同一算子类型。对于隐藏宽度 d ，每个分支以输入投影开始：

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

随后是 L 个隐藏层和读出：

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

2.3.1 单分支和节点级残差变体

PlainGNN. 普通基线堆叠 L 层相同算子，不含残差复用：

$$\mathbf{H}^{(\ell+1)} = \psi(\text{ReLU}(\Phi_{\ell}(\mathbf{H}^{(\ell)}, \mathbf{A}))). \quad (7)$$

NodeResGNN. 第一个残差变体在同一分支内保存前一隐藏状态：

$$\mathbf{H}^{(\ell+1)} = \psi(\text{ReLU}(\Phi_{\ell}(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}))). \quad (8)$$

NodeCrossGNN. 节点级交叉残差模型维护两个具有独立输入投影的并行分支：

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}^{(1)}\left(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (9)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}^{(2)}\left(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (10)$$

最终节点表示为两分支之和：

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (11)$$

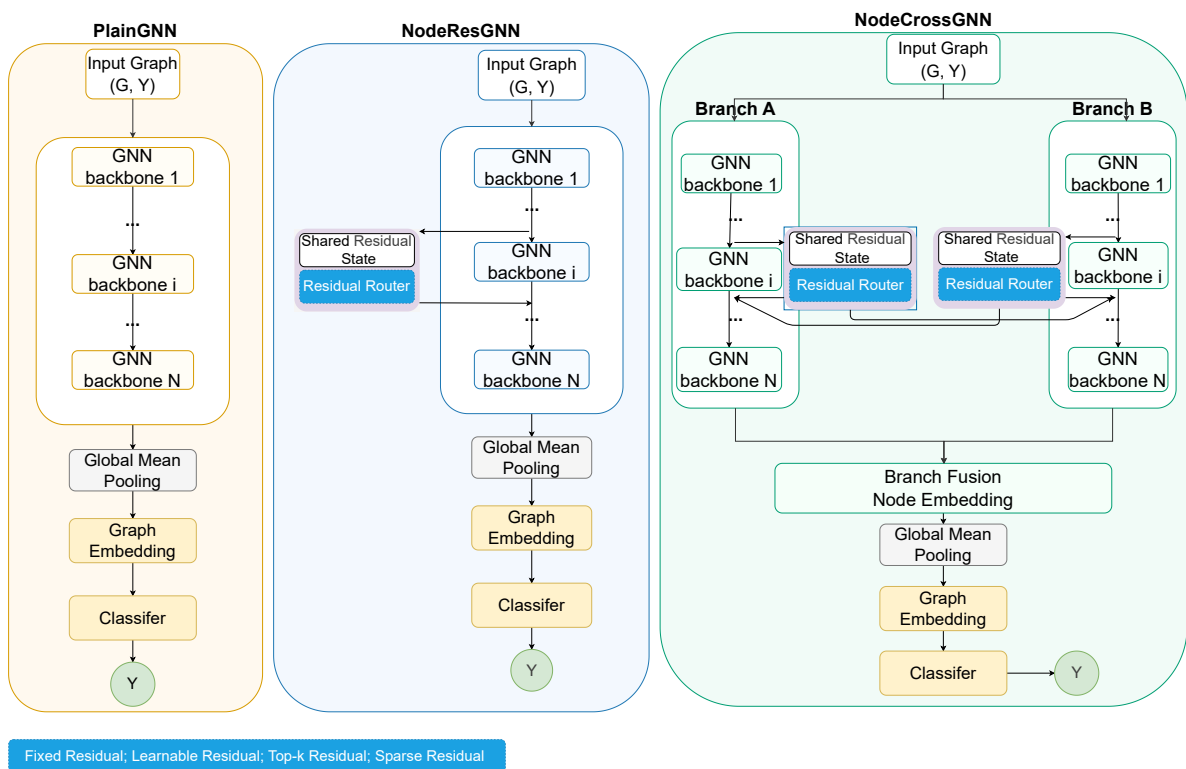


图 1: 节点级 CR-GNN 架构家族。左: PlainGNN——无状态复用的顺序消息传递层。中: NodeResGNN——单分支残差复用, 每层接收前一隐藏状态作为额外输入。右: NodeCrossGNN——双分支交叉残差交换, 每个分支复用自身和对侧的先前隐藏状态; 两分支求和后进行全局均值池化和分类。

2.3.2 图级残差传播

GraphResGNN. 该变体链接三个图条件块：

$$\mathbf{g}^{(t+1)} = B_{t+1}(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}), \quad t \in \{1, 2\}. \quad (12)$$

GraphCrossGNN. 四个块被组织为两个顺序对。在每个阶段内，两个块的输出被交换并作为下一阶段的图级输入复用：

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}), B_{2t}(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}) \right), \quad (13)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = (\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)}). \quad (14)$$

最终图嵌入为：

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (15)$$

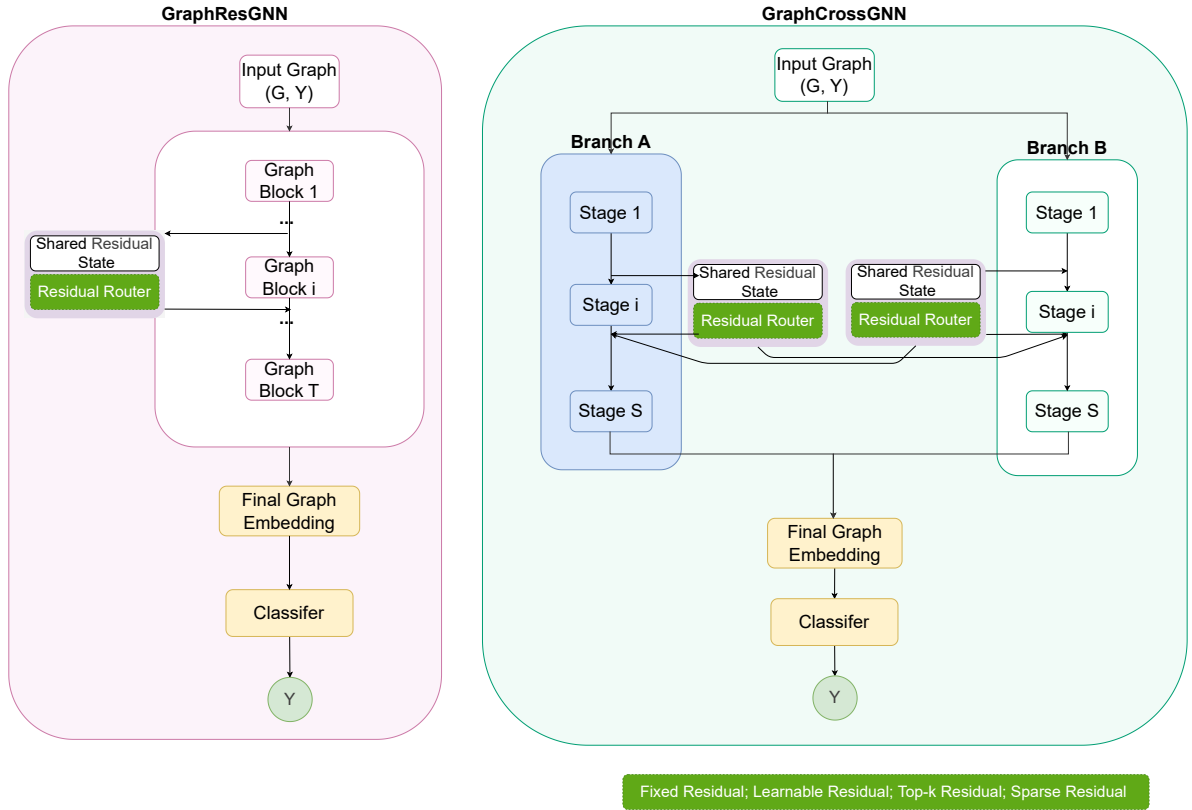


图 2: 图级 CR-GNN 架构家族。左: GraphResGNN——顺序图条件块, 共享残差状态通过残差路由器跨块传播。右: GraphCrossGNN——双分支交叉摘要交换, 两条并行块序列 (分支 A 和分支 B) 在配对阶段之间交换图级摘要。残差路由器支持四种路由模式: 固定、可学习、Top-k 和稀疏。

2.4 残差路由机制

除架构家族本身外，本研究还评估了残差信息如何被注入。这第二层分析是必要的，因为仅靠架构比较无法回答明显的残差优势究竟来自额外路径的存在，还是来自该路径被过滤的方式。针对性消融实验中使用了三种路由模式：

- **可学习路由**：稠密残差注入，由自适应门控调制。
- **Top-k 路由**：仅保留最强残差通道后再注入。
- **稀疏路由**：抑制弱残差系数，留下经过滤的残差路径。

这些路由模式不被用于重新定义核心架构家族，而是作为实验设计中选定残差和交叉残差目标之上的一个机制层。因此，路由研究是解释性而非装饰性的：它检验同一架构在残差信息以稠密、选择性或稀疏方式向前传递时是否表现不同。

2.5 数据集

为在具有生物学意义的设定下检验所提出的架构，基准测试被组织为两个层次。

主生物基准测试包含 PROTEINS、DD 和 ENZYMES (Fey and Lenssen, 2019; Morris et al., 2020)。补充鲁棒性测试集包含 MUTAG、AIDS 和 Mutagenicity。该组织方式使主生物学声明聚焦于蛋白质导向数据集，同时仍在额外分子图上检验结构鲁棒性。

PROTEINS PROTEINS 包含蛋白质的图表示，其中节点表示二级结构元素，边编码它们之间的邻域关系。任务为二值图分类。该数据集包含 1,113 张图，2 个类别，每节点 3 维输入特征。

DD DD 是一个蛋白质结构数据集，其中每张图对应一个蛋白质，节点表示氨基酸或结构相关单元，通过邻近性导出的边连接。任务为二值分类。DD 包含 1,178 张图，2 个类别，89 维输入特征。

ENZYMES ENZYMES 是一个 6 类酶分类基准，包含 600 张图和 3 维输入特征。它在保持酶导向生物学解释的同时，仍可与研究中其他图分类基准直接比较。

所有数据集通过统一基准接口使用 (Fey and Lenssen, 2019; Morris et al., 2020)。我们保持预处理尽可能轻量，不引入任务特定的特征工程、图增强或边重设计。

所有报告结果遵循相同的分层两阶段评估设计：

- **外评估**：在图级别的分层五折交叉验证
- **内验证**：从训练折中抽取的分层验证子集，验证比例为 0.1

表 1: Unified summary of the main biological benchmark package.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	5-fold CV + val	600	6	3	32.6 / 62.1	main

表 2: Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

主要评估指标为保留测试折上的图分类准确率。对于折 k 和测试集 $\mathcal{T}_k$:

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (16)$$

跨五个折, 我们报告平均准确率:

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (17)$$

和对应的标准差:

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (18)$$

2.6 比较方法与主干算子

基准评估五种架构, 在四种消息传递算子 (GCNConv、GATConv、SAGEConv、GINConv) 下进行。

每种方法在四种消息传递算子下评估:

- GCNConv
- GATConv
- GINConv
- SAGEConv

2.7 训练配置与实现细节

所有模型均在 PyTorch Geometric (Fey and Lenssen, 2019) 中实现，并在统一协议下训练。关键超参数汇总于表 3。训练采用早停机制：当验证损失连续 60 轮未改善时终止训练，并保留验证损失最低的检查点用于测试评估。不应用批归一化或层归一化；唯一正则化为 dropout 和权重衰减。所有训练-验证-测试划分由全局随机种子 1024 固定，确保每个模型在每折内看到相同的数据划分。不应用任务特定特征工程、图增强或边重设计；直接使用 TUDataset 的原始节点特征和邻接结构。

表 3: 所有实验共享的关键训练超参数。

Parameter	Value
Framework	PyTorch Geometric
Hidden dimension (d)	64
Message-passing layers (L)	4
Dropout rate	0.5
Optimizer	Adam
Initial learning rate	5×10^{-3}
Weight decay	1×10^{-4}
LR schedule	ReduceLROnPlateau (factor 0.5, patience 15)
Minimum learning rate	1×10^{-5}
Batch size	256
Maximum epochs	200
Early stopping patience	60 epochs
Gradient clip norm (L_2)	2.0
Pooling	Global mean pooling
Loss function	Cross-entropy
Random seed	1024
Validation split ratio	0.1

残差和交叉残差变体中使用的可学习门控对初始化为 0.8 的标量参数应用 sigmoid 激活，允许每次残差注入在训练过程中被自适应缩放。对于 Top-k 路由模式，仅保留残差通道（按绝对值确定）中最强比例，再进行注入。对于稀疏路由模式，阈值为 λ 的 softshrink 算子将幅值低于 λ 的残差系数抑制为零，留下经过滤的残差路径。

2.8 实验设计

实证研究将确认性五折证据与探索性分析分开。基准评估了六个数据集、五种架构、四种消息传递算子和五外折，统一在一个协议下。基准分为一个生物学核心 (PROTEINS、DD、ENZYMES) 和一个补充鲁棒性测试集 (MUTAG、AIDS、Mutagenicity)。

针对性消融实验围绕引言中提出的三个研究问题，通过受控机制实验展开：

1. **Q1: 交叉残差交互何时有帮助？** 交叉门控消融。在交叉架构已具备局部竞争力的六个设置中，评估四种门控配置（可学习、固定 0.0、固定 0.5、固定 1.0）。这检验了所观察到的交叉残差优势是否依赖于自适应注入控制，还是任何固定权重的额外路径都能产生类似增益。该比较隔离了交叉残差收益的来源：自适应过滤还是结构性通路添加。
2. **Q2: 普通残差复用何时依然保持更强的默认选择？** 门控消融。在最强图级残差目标（AIDS + GraphResGNN + GINConv）上，比较相同条件下的可学习门控与固定强度门控。这直接检验了残差家族的默认优势是来自自适应复用，还是仅来自拥有额外注入路径，以及固定残差注入是否能匹配或超越自适应控制。
3. **Q3: 残差路由设计如何改变答案？** 残差路由消融。在四个代表性目标（涵盖残差导向和交叉导向设置）上比较可学习稠密路由、三种保留比例（0.25, 0.5, 0.75）的 Top-k 通道选择和三种阈值（0.02, 0.05, 0.10）的稀疏抑制，均采用相同五折协议。这将分析从架构标签——模型被称为“残差”还是“交叉残差”——推进到机制层面：历史信息在复用前应被多激进地过滤。

残差参数扫描通过对相同代表性目标变化 Top-k 比例和稀疏强度，扩展了路由分析。单折深度、dropout 和学习率敏感性扫描作为探索性证据报告，不用于支撑主要声明。五折结果贯穿全文作为确认性证据。

2.9 评估指标与统计报告

主要指标为保留测试折上的图分类准确率。主基准和针对性消融结果以五折平均准确率与标准差报告。五折实验提供确认性证据；单折参数扫描作为探索性分析报告。

3 结果

所有主结果均以五折平均最佳测试准确率 $\pm$ 折级标准差报告；单折敏感性分析仅作为探索性证据。

3.1 整体基准性能

基准评估在 GCNConv 下对六个数据集和五种 CR-GNN 架构进行了五折评估。表 4（生物核心）和表 5（补充鲁棒性）共同显示，没有单一架构能够主导所有数据集。GraphResGNN 在 ENZYMES、AIDS 和 Mutagenicity 上最强；NodeResGNN 在 PROTEINS 和 MUTAG 上领先。DD 是唯一一个交叉残差变体同时优于两种残差对照的数据集。

表 4: Main biological benchmark results. Cells show mean $\pm$ std [min, max] over five folds. Bold: best mean per dataset.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7026 $\pm$ 0.0331 [0.655, 0.735]	0.6970 $\pm$ 0.0421 [0.646, 0.763]	0.2683 $\pm$ 0.0509 [0.217, 0.358]
NodeResGNN	0.7143 $\pm$ 0.0223 [0.685, 0.744]	0.7088 $\pm$ 0.0332 [0.651, 0.754]	0.2683 $\pm$ 0.0517 [0.217, 0.367]
NodeCrossGNN	0.6945 $\pm$ 0.0488 [0.613, 0.735]	0.7164 $\pm$ 0.0315 [0.677, 0.771]	0.2717 $\pm$ 0.0692 [0.167, 0.367]
GraphResGNN	0.7071 $\pm$ 0.0402 [0.655, 0.749]	0.7275 $\pm$ 0.0144 [0.711, 0.747]	0.3333 $\pm$ 0.1000 [0.150, 0.425]
GraphCrossGNN	0.7017 $\pm$ 0.0520 [0.608, 0.757]	0.7207 $\pm$ 0.0322 [0.694, 0.784]	0.2850 $\pm$ 0.0868 [0.167, 0.408]

表 5: Supplementary robustness benchmark results. Cells show mean $\pm$ std [min, max] over five folds. Bold: best mean per dataset.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7343 $\pm$ 0.0433 [0.684, 0.811]	0.8295 $\pm$ 0.0456 [0.755, 0.877]	0.7830 $\pm$ 0.0170 [0.764, 0.812]
NodeResGNN	0.7451 $\pm$ 0.0504 [0.684, 0.838]	0.8615 $\pm$ 0.0314 [0.800, 0.885]	0.7874 $\pm$ 0.0142 [0.769, 0.809]
NodeCrossGNN	0.7398 $\pm$ 0.0523 [0.684, 0.838]	0.8850 $\pm$ 0.0380 [0.818, 0.930]	0.7939 $\pm$ 0.0196 [0.770, 0.826]
GraphResGNN	0.7238 $\pm$ 0.0778 [0.649, 0.865]	0.9090 $\pm$ 0.0194 [0.890, 0.935]	0.8022 $\pm$ 0.0244 [0.763, 0.832]
GraphCrossGNN	0.7397 $\pm$ 0.0531 [0.684, 0.838]	0.8740 $\pm$ 0.0460 [0.800, 0.943]	0.7992 $\pm$ 0.0232 [0.756, 0.826]

3.2 各种复用拓扑分别在何时发挥作用？

数据集层面的结果揭示出两种截然不同的制度，在不同制度下各有不同的复用拓扑表现出色。

制度 1——图级上下文具有判别力。在 PROTEINS、AIDS 和 Mutagenicity 上，GraphResGNN 和 GraphCrossGNN 占据了前两位。这些数据集共享一个共同属性：全局图级结构携带了强烈的判别信号。GraphResGNN 的顺序块设计在每个阶段重新注入池化后的图摘要，恰好适合这一制度。在 AIDS 上，它取得了所有架构在所有数据集上的最高准确率 (0.9090) 和最低方差 (± 0.0194)。

制度 2——局部结构基序具有判别力。在 MUTAG 上——平均仅 18 个节点的小分子图——NodeResGNN 和 NodeCrossGNN 是最强的 CR-GNN 变体，而 GraphResGNN 排名最末。相同的模式出现在 DD 上，即核心数据集中最大且最稀疏的蛋白质图：两个交叉残差变体均优于两个残差变体。在这些设定下，激进的图级表示重塑似乎抑制了最为关键的局部结构信号。

ENZYMES 则独立于上述两种制度之外：所有架构的表现均不佳（最佳准确率 0.3333，仅比 6 类随机基线高出约 16 个百分点），反映出在仅有 600 张图、3 维特征和 6 个类别的设定下学习的困难。

3.3 机制分析：表示连续性与性能之间的张力

为什么 GraphResGNN——一个刻意在块间改变表示的架构——能够优于那些更忠实地保存表示的架构？为回答这一问题，我们在 PROTEINS 数据集（GCNConv 算子）上追踪了从 $L = 1$ 到 $L = 8$ 层的隐藏状态相似性（CKA 和余弦相似性）以及逐层梯度

表 6: Winner summary with max/min fold accuracy and parameter count for each dataset.

Dataset	Winner	Mean Acc	Max Acc	Min Acc
PROTEINS	NodeResGNN	0.7143	0.7444	0.6847
DD	GraphResGNN	0.7275	0.7468	0.7106
ENZYMES	GraphResGNN	0.3333	0.4250	0.1500
MUTAG	NodeResGNN	0.7451	0.8378	0.6842
AIDS	GraphResGNN	0.9090	0.9350	0.8900
Mutagenicity	GraphResGNN	0.8022	0.8316	0.7629

范数。

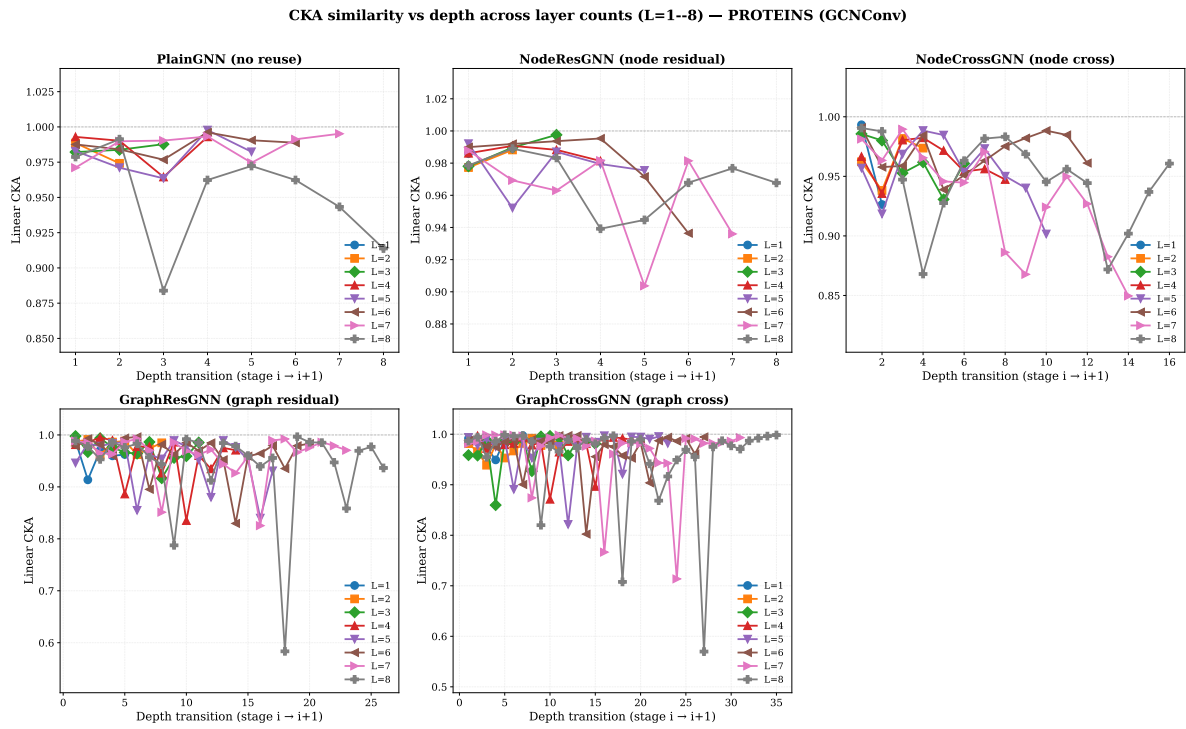


图 3: $L = 1$ 至 $L = 8$ 各层数下相邻深度阶段之间的 CKA 相似性。每个子图展示一种模型。

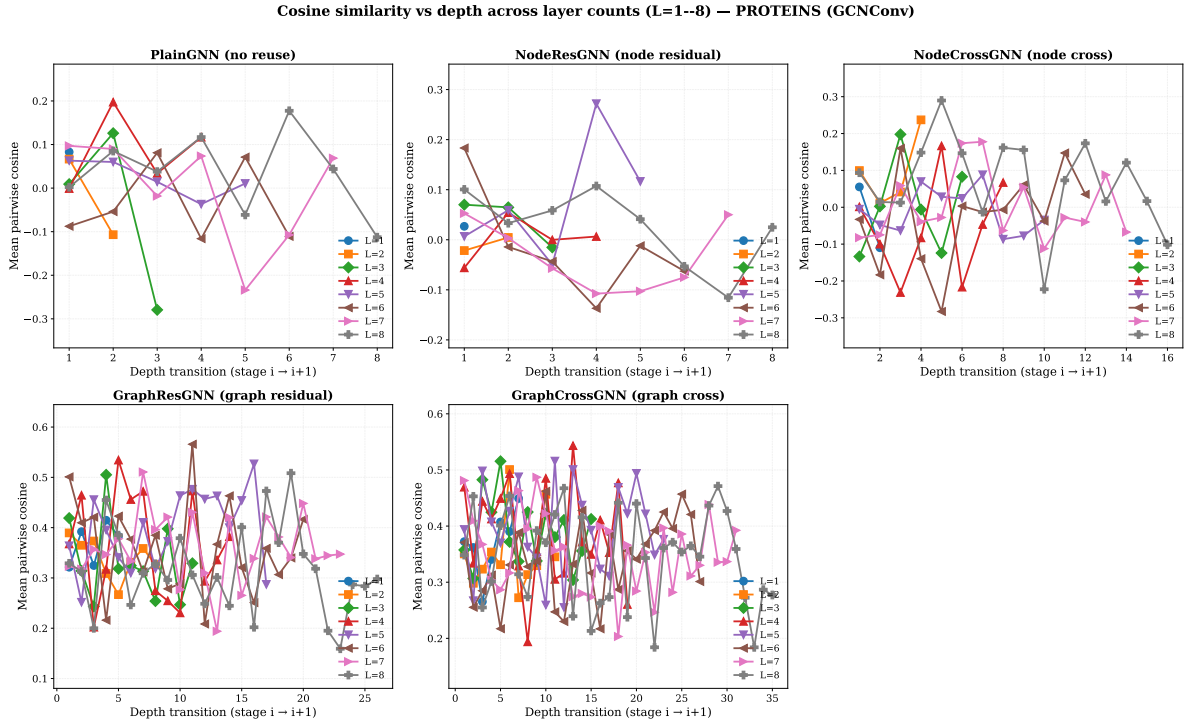


图 4: $L = 1$ 至 $L = 8$ 各层数下相邻深度阶段之间的余弦相似性。

图 3 和 4 揭示了一个反直觉的模式。在 $L = 8$ 时, CKA 的排序为 NodeResGNN (0.968) > GraphCrossGNN (0.952) > PlainGNN (0.951) > NodeCrossGNN (0.946) > GraphResGNN (0.942)。最忠实地保存表示的架构 (NodeResGNN) 仅在 24 个组合中的 3 个上获胜; 最激进地重塑表示的架构 (GraphResGNN) 则赢得了 24 个中的 12 个。我们将此称为 **CKA-性能悖论**: 更强的逐层表示连续性并不能预测更高的最终准确率。

这一悖论具有结构性的解释。GraphResGNN 的三个图条件块每个都接收前一块的池化图嵌入作为条件信号。这种精心设计的结构化重塑——在每一阶段重新注入不断累积的全局上下文——之所以能产生更具判别力的最终表示, 恰恰是因为表示在块间发生了实质性变化。相反, NodeResGNN 对表示在深度方向上的忠实保存可能限制了其适应任务特定结构的能力。

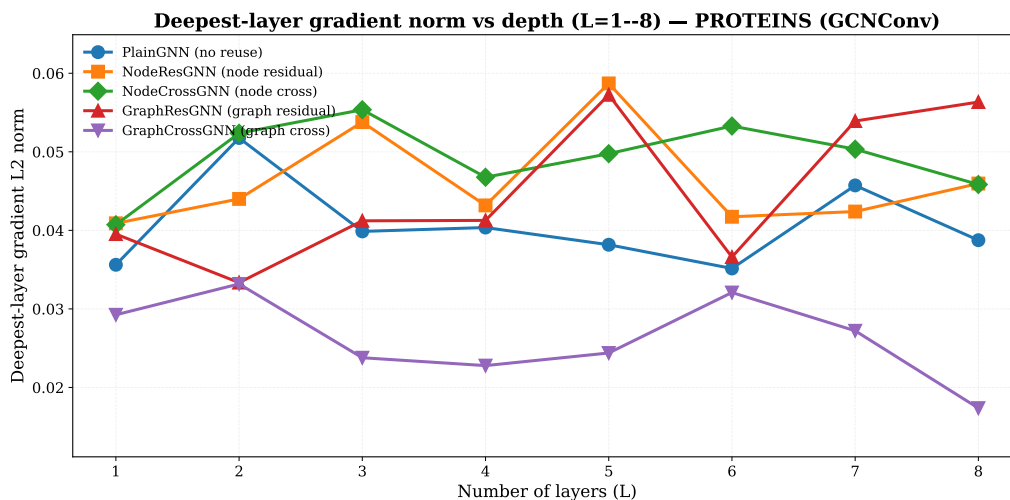


图 5: 最深层梯度 L2 范数随 $L = 1-8$ 层数的变化 (PROTEINS, GCNConv)。

图 5 证实了 PlainGNN 的最深层梯度范数随深度增加而退化最快；残差家族在整个深度范围内保持了更稳定的梯度。NodeResGNN 和 NodeCrossGNN 表现出相当的梯度稳定性，确认了跨分支交换不会引入梯度干扰。

3.4 量化性能权衡

为超越定性模式，表 7 统计了每种架构在全部 24 个数据集-算子组合中的获胜次数，表 8 衡量了跨数据集排名稳定性。

表 7: 全部 24 个数据集-算子组合上的获胜次数 (GCNConv)。

Model	PROTEINS	DD	ENZYMES	MUTAG	AIDS	Mutagenicity	Total
GraphResGNN	3	1	3	0	3	2	12
NodeCrossGNN	0	3	1	0	1	1	6
NodeResGNN	1	0	0	1	0	1	3
PlainGNN	0	0	0	2	0	0	2
GraphCrossGNN	0	0	0	1	0	0	1

权衡现在被量化了。GraphResGNN 获胜最频繁 (12/24)，但排名稳定性最弱 (标准差 1.46) ——它可能大胜，也可能惨败。GraphCrossGNN 仅获胜一次，却是最稳定的表现者 (标准差 0.75)，从未低于第四名。NodeCrossGNN 居于中间：第二多的获胜次数 (6/24)、中等稳定性 (标准差 1.00)，以及在 DD 上具有独特优势。

3.5 探索性敏感性分析

在 PROTEINS、DD 和 ENZYMES 上针对深度、dropout 和学习率的辅助敏感性检验不会实质性改变上述结论。

表 8: 六个数据集上的平均排名与排名稳定性 (GCNConv)。

Model	Mean rank	Gap to best	Rank std
GraphResGNN	1.8	0.005	1.46
GraphCrossGNN	2.7	0.018	0.75
NodeCrossGNN	3.0	0.022	1.00
NodeResGNN	3.2	0.024	1.57
PlainGNN	4.3	0.036	0.75

4 讨论

上述结果提出了若干简单的架构排名无法回答的问题。本节直接回答这些问题。

4.1 为什么表示连续性最弱的架构获胜最频繁？

CKA-性能悖论——GraphResGNN 在深度处的 CKA 最低 (0.942) 却获胜最多 (12/24)，而 NodeResGNN 的 CKA 最高 (0.968) 但在复用变体中获胜最少 (3/24)——具有结构性的解释。GraphResGNN 链接了三个图条件块，每个块接收前一块的池化图嵌入作为条件输入。这一设计使用不断累积的全局上下文在每一阶段刻意重塑表示：它的目标不是保存表示，而是逐步精化表示。在全局结构具有判别力的数据集上 (AIDS、PROTEINS)，这一策略表现出色。在局部基序占主导的 MUTAG 上，它反而失效 (排名 5/5)。

NodeResGNN 忠实保存表示，但代价是：层间变化过小的表示可能无法适应任务特定的结构。交叉残差架构解决了这一张力：双分支交换为单分支可能抑制的信息提供了替代通路，且无需强制执行使 GraphResGNN 变得脆弱的激进重塑。这正是为什么 GraphCrossGNN 实现了最一致的排名 (标准差 0.75)，以及为什么 NodeCrossGNN 在 DD 上占据统治地位——互补通路保存在该数据集上尤为重要。

回答：当全局上下文具有判别力时，激进的表示重塑能产生收益；但当局部基序重要时，这会造成脆弱性。交叉残差架构在连续性与变化之间取得平衡，实现了最一致的性能。

4.2 何时应选择交叉残差而非普通残差？

基准测试识别出一个清晰的、交叉残差占主导的应用场景：大型稀疏蛋白质图。在 DD 上 (1,178 张图, 89 维特征)，NodeCrossGNN 在四种算子中的三种下获胜。在此设定下，单条传播分支会在重复消息传递中倾向于抑制稀有但具有判别力的结构信号；双分支节点级交换则提供了互补的并行通路来保存这些信号。在基于 GATConv 的其他配置 (AIDS、Mutagenicity、ENZYMES) 上也能观察到相同的模式：NodeCrossGNN 始终具有竞争力。

相反，在全局上下文具有强烈判别力的更小、更稠密的图上 (AIDS、PROTEINS)，GraphResGNN 的顺序块设计是更强的选择。

回答：对于大型稀疏图，尤其是配合基于注意力的算子时，选择交叉残差 (NodeCrossGNN)。对于全局上下文占主导的更小、更稠密的图，选择图级残差 (GraphResGNN)。

4.3 路由策略在架构选择之外是否重要？

是的。两项证据支持这一结论。第一，门控消融显示，在最强的图级残差目标 (AIDS + GraphResGNN + GINConv) 上，可学习门控优于固定替代方案，但差距很小——表明自适应控制有帮助，但并非决定性因素。在六个交叉偏好目标上，可学习门控与固定门控平分秋色 (各胜三组)，表明交叉残差架构对门控配置具有鲁棒性，与其更高的排名稳定性一致。

第二，更重要的是，路由模式的选择具有实质性影响。稀疏路由 (阈值 0.05) 在四个代表性目标中的三个上最优；可学习稠密路由仅在最强的图级残差目标上保持最佳。三个偏好稀疏的目标中有两个涉及交叉残差架构，揭示了一种机制层面的协同效应：稀疏性在弱残差通道随深度累积为噪声之前将其抑制，而跨分支通路则通过独立路径保存真正互补的信号。激进的稀疏 (0.10) 损害了图级残差目标，确认了该效应具有阈值依赖性。

配对检验矩阵 (表 14-15) 确认了大多数架构级差异相对于折间变异是有限的：差异是真实的，但具有条件性。

回答：路由策略是一个有意义的次要设计轴。稀疏路由搭配交叉残差是最频繁的最优组合；可学习稠密路由搭配图级残差在追求已充分了解的数据集上的峰值准确率时表现最佳。

4.4 局限性

若干局限性制约了这些发现的普适性。第一，统计功效有限：仅有五折数据时，只有大规模效应才能达到 $p < 0.05$ 。第二，CKA-性能悖论仅在一个数据集-算子组合 (PROTEINS, GCNConv) 上得到记录；该悖论能否泛化仍是一个开放问题。第三，基准数据集为中等规模生物分子图；在更大或不同类型的图上，所述权衡可能发生变化。第四，路由分析覆盖了四个目标——足以揭示模式，但不足以声称其具有普适性。

4.5 未来方向

最重要的扩展是在更大规模基准和更丰富的输入上检验 CKA-性能悖论。如果该悖论能够泛化，将促使领域重新审视深度 GNN 架构的评估方式——将关注点从逐层稳定性指标转移到表示变化的结构性上。稀疏路由加交叉残差的协同效应也应在本文研究的四个目标之外得到更广泛的检验。

表 9: AIDS gate ablation on GraphResGNN + GINConv. Numbers are five-fold mean accuracy $\pm$ standard deviation. Bold: best per column.

Gate setting	Mean accuracy
Learnable	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9605 $\pm$ 0.0144
Fixed 1.0	0.9480 $\pm$ 0.0300

表 10: Fold-level AIDS gate ablation on AIDS + GraphResGNN + GINConv. Bold: best per column.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean $\pm$ Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 $\pm$ 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 $\pm$ 0.0300

5 结论

本文将 CR-GNN 作为一个受控的生物分子图分类框架加以研究，并发现该设计空间揭示了峰值准确率、跨数据集一致性和表示连续性之间的系统性权衡。主要贡献不是一个普遍占优的架构，而是一个使这些权衡定量可见并提供架构选择实践指导的可复现比较框架。

关键实证发现如下。第一，在 CR-GNN 家族 (GCNConv) 中，GraphResGNN 最频繁地实现了最高的峰值准确率 (24 个数据集-算子组合中的 50%)，但深层表示连续性最弱 ($L = 8$ 时 CKA 0.942)，排名波动最大 (标准差 1.46)。第二，NodeCrossGNN 是五种拓扑中第二强的架构 (25% 的组合)，在 DD 上跨越三种算子占统治地位，且在大而稀疏的图与基于注意力的算子组合上特别有效。第三，GraphCrossGNN 是 CR-GNN 架构中排名最一致的设计 (标准差 0.75)，从未低于第四名，使其在数据集特征不确定时是最安全的架构选择。第四，深度扩展分析记录了一个“CKA-性能悖论”：表示连续性最高的架构 (NodeResGNN, CKA 0.968) 在复用变体中获胜最少，而连续性最低的架构 (GraphResGNN) 获胜最多。这挑战了“更强的表示稳定性必然产生更好的性能”这一假设。

路由分析提供了机制层面的证据，表明稀疏路由 (阈值 0.05) 在四个代表性目标中的三个上最优，其中两个涉及交叉残差架构。这种协同效应——稀疏性抑制弱通道，而跨分支交换保存互补信息——值得作为未来 GNN 架构发展的候选设计策略进一步研究。

这些结论的范围应保持严谨。最有力的声明得到分层五折基准、针对性五折消融以及在 PROTEINS (GCNConv) 上从 $L = 1$ 到 $L = 8$ 的深度扩展分析的支持。统计证据 (表 14 和表 15) 确认了架构级效应相对于折间变异是有限的，这与本文以机制分析

表 11: Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy $\pm$ standard deviation. Bold: best per row.

Dataset	Target	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCross + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCross + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCross + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCross + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCross + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCross + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

表 12: Residual-routing and parameter-sweep summary on four representative targets (transposed). Numbers are five-fold mean accuracy $\pm$ standard deviation. Bold: best per column.

Routing	AIDS+GraphRes+GIN	PROT+GraphRes+SAGE	AIDS+NodeCross+GAT	DD+NodeCross+GAT
Learnable	0.9500 $\pm$ 0.0087	0.7178 $\pm$ 0.0411	0.8970 $\pm$ 0.0491	0.7080 $\pm$ 0.0172
Top-k 0.25	0.9075 $\pm$ 0.0237	0.7241 $\pm$ 0.0407	0.9160 $\pm$ 0.0128	0.7062 $\pm$ 0.0249
Top-k 0.50	0.9330 $\pm$ 0.0251	0.7151 $\pm$ 0.0397	0.8960 $\pm$ 0.0483	0.7029 $\pm$ 0.0160
Top-k 0.75	0.9240 $\pm$ 0.0287	0.7214 $\pm$ 0.0318	0.8960 $\pm$ 0.0489	0.7164 $\pm$ 0.0248
Sparse 0.02	0.9470 $\pm$ 0.0181	0.7160 $\pm$ 0.0248	0.9060 $\pm$ 0.0268	0.7147 $\pm$ 0.0259
Sparse 0.05	0.9250 $\pm$ 0.0152	0.7259 $\pm$ 0.0475	0.9180 $\pm$ 0.0162	0.7181 $\pm$ 0.0196
Sparse 0.10	0.8315 $\pm$ 0.0549	0.7088 $\pm$ 0.0521	0.9025 $\pm$ 0.0139	0.7105 $\pm$ 0.0243
Best	Learnable	Sparse 0.05	Sparse 0.05	Sparse 0.05

为核心而非排行榜竞赛的定位一致。本研究涉及中等规模生物分子基准上的受控图分类行为，其本身并不确立大规模可扩展性或直接生物学可解释性。

在实际层面，本工作提供以下指导：当特征明确的数据集上的峰值准确率是优先事项时，选择 GraphResGNN 搭配可学习稠密路由；当需要跨多样数据集的稳定可预测性能时，选择 GraphCrossGNN 搭配稀疏路由；当处理大而稀疏的图时，选择 NodeCrossGNN 搭配 GATConv。这些结果将 CR-GNN 定位为一个实用设计空间，并为未来在更丰富的生物学输入、更大规模基准以及本文所记录权衡可被进一步检验和精化的图学习设定上的工作提供了方法论基础。

6 致谢

作者感谢开源图学习社区以及本研究所用基准资源的维护者。同时感谢审稿人和编辑的时间与反馈。

参考文献

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.

表 13: Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphRes + GIN	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphRes + SAGE	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCross + GAT	0.8970	0.9160	0.9180	Sparse
DD + NodeCross + GAT	0.7080	0.7164	0.7181	Sparse

Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.

Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.

Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning (ICML)*, pages 1725–1735, 2020a.

Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020b.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.

Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.

William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

表 14: Paired t -tests for main biological benchmark datasets. Cells: Δ (Cohen's d , p).
Bold: $p < 0.05$ ($n=5$ folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	NodeRes – Plain	+0.012 (+0.55, 0.286)	+0.000 (+0.00, 0.999)	+0.006 (+0.31, 0.524)	+0.012 (+0.56, 0.277)
	NodeCross – Plain	-0.008 (-0.28, 0.571)	-0.011 (-0.22, 0.647)	-0.002 (-0.07, 0.881)	-0.008 (-0.23, 0.632)
	GraphRes – Plain	+0.004 (+0.45, 0.375)	+0.015 (+0.48, 0.345)	+0.030 (+1.69, 0.019)	+0.040 (+1.10, 0.069)
	GraphCross – Plain	-0.001 (-0.02, 0.961)	-0.008 (-0.24, 0.623)	+0.004 (+0.24, 0.622)	+0.004 (+0.16, 0.743)
	NodeCross – NodeRes	-0.020 (-0.53, 0.302)	-0.011 (-0.19, 0.690)	-0.008 (-0.72, 0.181)	-0.020 (-0.87, 0.125)
	GraphCross – GraphRes	-0.005 (-0.15, 0.751)	-0.023 (-2.18, 0.008)	-0.026 (-1.47, 0.031)	-0.036 (-2.36, 0.006)
	GraphRes – NodeRes	-0.007 (-0.26, 0.588)	+0.015 (+0.37, 0.459)	+0.023 (+0.75, 0.171)	+0.028 (+1.05, 0.079)
	NodeCross – GraphCross	-0.007 (-0.49, 0.337)	-0.003 (-0.10, 0.838)	-0.005 (-0.21, 0.659)	-0.012 (-0.39, 0.427)
DD	NodeRes – Plain	+0.012 (+0.39, 0.429)	+0.002 (+0.24, 0.624)	+0.016 (+0.48, 0.347)	+0.001 (+0.03, 0.946)
	NodeCross – Plain	+0.019 (+0.55, 0.286)	+0.035 (+0.90, 0.115)	+0.023 (+0.87, 0.124)	+0.025 (+0.93, 0.107)
	GraphRes – Plain	+0.030 (+0.64, 0.223)	+0.020 (+0.50, 0.326)	+0.021 (+0.91, 0.112)	+0.002 (+0.06, 0.892)
	GraphCross – Plain	+0.024 (+0.87, 0.123)	+0.019 (+0.46, 0.357)	+0.004 (+0.19, 0.699)	+0.008 (+0.20, 0.672)
	NodeCross – NodeRes	+0.008 (+0.43, 0.395)	+0.033 (+0.88, 0.121)	+0.007 (+0.64, 0.226)	+0.024 (+0.94, 0.103)
	GraphCross – GraphRes	-0.007 (-0.23, 0.637)	-0.002 (-0.11, 0.815)	-0.017 (-2.82, 0.003)	+0.006 (+0.14, 0.762)
	GraphRes – NodeRes	+0.019 (+0.64, 0.228)	+0.019 (+0.46, 0.359)	+0.005 (+0.24, 0.624)	+0.001 (+0.03, 0.954)
	NodeCross – GraphCross	-0.004 (-0.31, 0.526)	+0.016 (+1.07, 0.075)	+0.019 (+1.53, 0.027)	+0.017 (+0.57, 0.274)
ENZYMES	NodeRes – Plain	-0.000 (-0.00, 1.000)	+0.005 (+0.19, 0.697)	-0.017 (-0.94, 0.103)	+0.030 (+0.47, 0.351)
	NodeCross – Plain	+0.003 (+0.09, 0.847)	+0.032 (+0.45, 0.369)	+0.007 (+0.24, 0.614)	+0.032 (+2.12, 0.009)
	GraphRes – Plain	+0.020 (+0.92, 0.107)	+0.020 (+0.32, 0.507)	+0.020 (+0.47, 0.360)	+0.068 (+2.45, 0.002)
	GraphCross – Plain	+0.020 (+0.55, 0.280)	+0.007 (+0.08, 0.875)	+0.020 (+0.33, 0.502)	+0.028 (+0.52, 0.303)
	NodeCross – NodeRes	+0.003 (+0.08, 0.878)	+0.027 (+0.36, 0.471)	+0.023 (+0.42, 0.399)	+0.002 (+0.07, 0.884)
	GraphCross – GraphRes	-0.000 (-0.01, 0.981)	-0.013 (-0.21, 0.672)	+0.000 (+0.00, 1.000)	-0.040 (-0.62, 0.237)
	GraphRes – NodeRes	+0.020 (+0.57, 0.264)	+0.015 (+0.28, 0.562)	+0.037 (+0.82, 0.151)	+0.038 (+1.02, 0.079)
	NodeCross – GraphCross	-0.017 (-0.47, 0.355)	+0.025 (+0.68, 0.211)	-0.013 (-0.40, 0.444)	+0.003 (+0.07, 0.894)

Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.

Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.

Christopher Morris, Roger Wattenhofer, and Kristian Kersting. Tdataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.

Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe

表 15: Paired t -tests for supplementary robustness datasets. Cells: Δ (Cohen’s d , p).
Bold: $p < 0.05$ ($n=5$ folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
MUTAG	NodeRes – Plain	+0.011 (+0.29, 0.545)	+0.000 (+0.00, 1.000)	-0.016 (-0.40, 0.416)	-0.005 (-0.12, 0.812)
	NodeCross – Plain	+0.016 (+0.36, 0.477)	+0.005 (+0.09, 0.850)	+0.005 (+0.15, 0.758)	+0.016 (+0.34, 0.498)
	GraphRes – Plain	+0.005 (+0.09, 0.864)	+0.011 (+0.33, 0.513)	-0.011 (-0.10, 0.839)	-0.016 (-0.22, 0.670)
	GraphCross – Plain	-0.011 (-0.20, 0.683)	+0.027 (+0.59, 0.262)	-0.022 (-0.28, 0.576)	-0.005 (-0.16, 0.756)
	NodeCross – NodeRes	+0.005 (+0.10, 0.836)	+0.005 (+0.10, 0.845)	+0.022 (+0.33, 0.516)	+0.022 (+0.37, 0.464)
	GraphCross – GraphRes	-0.016 (-0.34, 0.515)	+0.016 (+0.37, 0.467)	-0.011 (-0.26, 0.613)	+0.011 (+0.25, 0.618)
	GraphRes – NodeRes	-0.005 (-0.12, 0.808)	+0.011 (+0.17, 0.742)	+0.005 (+0.08, 0.881)	-0.011 (-0.12, 0.824)
	NodeCross – GraphCross	+0.027 (+0.71, 0.186)	-0.022 (-0.61, 0.240)	+0.027 (+0.53, 0.310)	+0.022 (+0.48, 0.349)
AIDS	NodeRes – Plain	+0.023 (+1.19, 0.062)	+0.010 (+0.89, 0.112)	+0.018 (+0.86, 0.125)	+0.015 (+0.59, 0.261)
	NodeCross – Plain	+0.028 (+1.06, 0.083)	+0.018 (+0.80, 0.144)	+0.020 (+1.52, 0.022)	+0.018 (+0.67, 0.222)
	GraphRes – Plain	+0.070 (+2.41, 0.005)	+0.025 (+1.57, 0.041)	+0.028 (+2.45, 0.004)	+0.048 (+2.47, 0.004)
	GraphCross – Plain	+0.025 (+0.94, 0.089)	+0.013 (+1.10, 0.068)	+0.018 (+1.16, 0.067)	+0.023 (+0.86, 0.124)
	NodeCross – NodeRes	+0.005 (+0.41, 0.420)	+0.008 (+0.52, 0.311)	+0.003 (+0.14, 0.778)	+0.003 (+1.61, 0.005)
	GraphCross – GraphRes	-0.045 (-1.46, 0.043)	-0.013 (-0.66, 0.218)	-0.010 (-0.44, 0.398)	-0.025 (-0.87, 0.125)
	GraphRes – NodeRes	+0.047 (+2.62, 0.002)	+0.015 (+1.19, 0.063)	+0.010 (+0.46, 0.370)	+0.033 (+2.02, 0.014)
	NodeCross – GraphCross	-0.003 (-0.10, 0.848)	+0.005 (+0.44, 0.387)	+0.003 (+0.20, 0.682)	-0.005 (-0.16, 0.751)
Mutagenicity	NodeRes – Plain	+0.004 (+0.26, 0.597)	+0.011 (+0.95, 0.101)	+0.014 (+0.93, 0.106)	+0.013 (+0.83, 0.137)
	NodeCross – Plain	+0.011 (+0.68, 0.201)	+0.020 (+1.05, 0.079)	+0.013 (+0.86, 0.127)	+0.017 (+0.95, 0.101)
	GraphRes – Plain	+0.019 (+1.24, 0.050)	+0.008 (+0.57, 0.273)	+0.012 (+0.62, 0.237)	+0.018 (+0.83, 0.139)
	GraphCross – Plain	+0.016 (+0.85, 0.131)	+0.009 (+0.46, 0.366)	+0.012 (+0.49, 0.333)	+0.010 (+0.80, 0.147)
	NodeCross – NodeRes	+0.006 (+0.26, 0.588)	+0.009 (+0.69, 0.195)	-0.001 (-0.13, 0.787)	+0.003 (+0.33, 0.503)
	GraphCross – GraphRes	-0.003 (-0.30, 0.545)	+0.002 (+0.08, 0.874)	+0.000 (+0.00, 0.999)	-0.008 (-0.32, 0.513)
	GraphRes – NodeRes	+0.015 (+0.87, 0.125)	-0.003 (-0.27, 0.583)	-0.003 (-0.18, 0.714)	+0.005 (+0.23, 0.637)
	NodeCross – GraphCross	-0.005 (-0.35, 0.476)	+0.010 (+0.50, 0.324)	+0.001 (+0.10, 0.832)	+0.007 (+0.49, 0.334)

Vendrig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*, pages 4800–4810, 2018.

Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint arXiv:2009.10698*, 2020.

Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhaocheng Liu, Zhongyuan Wang,

Peng Li, Ming Sun, Yu Shi, et al. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.